

Informe de Auditoría de Código

Panel de Control de Clientes (React 19 & Vite 8)

Proyecto: ProyectoLyEP2026

Cátedra: Legislación y Ejercicio Profesional

Carrera: APU — FI UNJu

Auditor: Frontend Senior & Code Auditor

Fecha de Emisión: 10 de Septiembre de 2026

1 Resumen Ejecutivo

El prototipo analizado presenta una base funcional operativa construida sobre una pila tecnológica moderna (React 19 y Vite 8), logrando resolver de forma básica los flujos de autenticación simulada, navegación protegida y consumo de recursos externos. No obstante, el proyecto evidencia una alta deuda técnica concentrada en la ausencia de una capa de servicios unificada, la fragmentación del estado entre React Context y llamadas directas y desincronizadas a `localStorage`, y la falta de aislamiento en los estilos CSS mediante selectores globales que generan colisiones visuales. Asimismo, existen vulnerabilidades de robustez críticas como la exposición de credenciales y contraseñas en el frontend, fallos en la ejecución de ESLint por variables huérfanas y advertencias de Fast Refresh, y carencia de validaciones defensivas ante respuestas de red fallidas o datos incompletos. Para alcanzar estándares profesionales de mantenibilidad, seguridad y escalabilidad, el código base requiere una reestructuración arquitectónica orientada a Layouts, centralización de API con Axios, tipado/validación robusta de formularios y adopción de CSS Modules.

2 Tabla de Hallazgos Técnicos

ID	PROBLEMA / DEFECTO DETECTADO	ARCHIVO(S) AFECTADO(S)	DIMENSIÓN	IMPACTO	PROPUESTA DE SOLUCIÓN TÉCNICA
HLG-01	Exposición de credenciales y lógica sensible en cliente: Usuarios y contraseñas en texto plano empaquetados en el bundle del cliente; contraseña de clientes mostrada en vista de detalle.	<code>autorizacionesService.s.js</code> <code>DetalleCliente.jsx</code>	Calidad / Seg.	ALTO	Sustituir mock estático por servicio de autenticación asíncrono con tokens seguros; ocultar inmediatamente la visualización de contraseñas en la ficha del cliente.
HLG-02	Fragmentación y pérdida de fuente única de verdad en sesión: Login setea role en <code>localStorage</code> , <code>DetalleCliente</code> lo lee de allí y <code>cerrarSesion</code> nunca lo borra.	<code>Login.jsx</code> <code>DetalleCliente.jsx</code> <code>AutorizacionesContext.t.jsx</code>	Estado	ALTO	Centralizar el rol/sector en <code>AutorizacionesContext</code> . Eliminar clave role aislada y consumir siempre <code>useAutorizaciones</code> de forma reactiva.
HLG-03	Omisión de capa de servicios y consumo HTTP disperso: <code>clientesService.js</code> solo implementa <code>crearCliente</code> (Axios); las vistas usan <code>fetch</code> nativo duplicando endpoints base.	<code>clientesService.js</code> <code>ListaClientes.jsx</code> <code>DetalleCliente.jsx</code>	Arquitectura	ALTO	Unificar todas las peticiones en una instancia configurada de Axios con <code>baseUrl</code> e implementar métodos CRUD completos desacoplados de las vistas.
HLG-04	Fallo en ejecución de linter (ESLint con 4 errores bloqueantes): Variables no utilizadas y exportación combinada de contexto que invalida Fast Refresh en Vite.	<code>App.jsx</code> , <code>routes.jsx</code> <code>DetalleCliente.jsx</code> <code>AutorizacionesContext.t.jsx</code>	Calidad	ALTO	Eliminar imports y variables huérfanas (<code>useAutorizaciones</code> , <code>Navigate</code> , <code>error</code>); desacoplar definición de contexto de su Provider.
HLG-05	Crash de arranque por deserialización sin try/catch: <code>JSON.parse</code> se ejecuta sobre <code>localStorage</code> sin control defensivo; un valor corrupto causa White Screen of Death.	<code>AutorizacionesContext.t.jsx</code>	React / Estado	ALTO	Envolver lectura en bloque <code>try...catch</code> con fallback a <code>null</code> y remoción de clave dañada en almacenamiento.

ID	PROBLEMA / DEFECTO DETECTADO	ARCHIVO(S) AFECTADO(S)	DIMENSIÓN	IMPACTO	PROPUESTA DE SOLUCIÓN TÉCNICA
HLG-06	Aislamiento de estado entre alta de cliente y listado: FormCliente crea un nuevo cliente pero ListaClientes no es notificado, dejando la tabla desactualizada.	FormCliente.jsx ListaClientes.jsx	React / Estado	MEDIO	Implementar callback prop onClienteCreado(nuevo) o estado global/hook para anexar el registro creado directamente a la tabla activa.
HLG-07	Efectos asíncronos sin cancelación ('AbortController'): Peticiones pendientes no se abortan al desmontar el componente; DetalleCliente carece de .catch.	ListaClientes.jsx DetalleCliente.jsx	React	MEDIO	Agregar AbortController en la función de limpieza del useEffect y capturar errores de red evitando bloqueos permanentes de carga.
HLG-08	Filtrado reactivo vulnerable a caídas por datos nulos: Lectura directa de cliente.name.lastname sin verificar existencia provoca TypeError ante datos incompletos.	ListaClientes.jsx	React / Calidad	MEDIO	Incorporar encadenamiento opcional (cliente?.name?.lastname ?? '') y valores por defecto tolerantes a fallos.
HLG-09	Contaminación de CSS global (CSS Pollution) y uso de !important: Reglas globales sobre form, table, input que colisionan entre vistas e interfieren con Bootstrap.	login.css, listacientes.css formcliente.css	Calidad / UX	MEDIO	Migrar a CSS Modules (*.module.css) o nomenclatura BEM de ámbito cerrado; remover todas las declaraciones !important.
HLG-10	Estructura de Layout rígida con navegación expuesta: App.jsx monta cabecera y barra de navegación fijas incluso en el Login y páginas de error.	App.jsx Nav.jsx	Arquitectura / UX	MEDIO	Crear un componente MainLayout con <Outlet /> para rutas autenticadas y layout limpio para rutas públicas de acceso.
HLG-11	Métricas estáticas hardcodeadas y lógica muerta en Dashboard: Indicadores numéricos fijos en HTML; evaluación redundante de !admin en ruta ya protegida.	Dashboard.jsx	Arquitectura	MEDIO	Calcular métricas en tiempo real a partir del array de clientes y roles activos; eliminar bloque de formulario de login muerto.
HLG-12	Acción destructiva sin confirmación previa: El botón "Eliminar Cliente" ejecuta la eliminación inmediatamente con un solo clic accidental.	DetalleCliente.jsx	UX	MEDIO	Incorporar un modal interactivo de confirmación antes de disparar la petición HTTP DELETE.
HLG-13	Feedback intrusivo mediante alert() nativo: Interrupción de la experiencia del usuario con ventanas emergentes bloqueantes ante error de autenticación.	Login.jsx	UX	BAJO	Sustituir diálogo emergente por componente inline <Alert variant="danger"> de React-Bootstrap.
HLG-14	Página 404 vacía de interacción y guía: ErrorPage.jsx carece de diseño, contexto y botones para regresar al panel de control.	ErrorPage.jsx	UX	BAJO	Diseñar una vista 404 asistida con mensaje descriptivo y botón de redirección rápida a la página principal.
HLG-15	Importación anómala de estilos y ausencia de path aliases: Referencia relativa ../src/css/app.css en vez de alias limpio con Vite.	App.jsx vite.config.js	Calidad	BAJO	Configurar alias @ en Vite apuntando a /src y normalizar imports relativos.
HLG-16	Título y metadatos por defecto en archivo HTML: Etiqueta <title> conserva el identificador genérico pvtrabajointegradorgrupo5.	index.html	Calidad / UX	BAJO	Actualizar el título a "Panel de Control de Clientes" y agregar metadatos semánticos en el encabezado.

3 Catálogo General de Mejoras Propuestas

A. Arquitectura y Enrutamiento

1. Implementación del Patrón Layout con React Router 7 (`<Outlet />`)

Resuelve: HLG-10

Descripción: Crear un componente `MainLayout` que agrupe `Header`, `Nav`, el contenedor de contenido mediante `<Outlet />` y `Footer`. En `routes.jsx`, estructurar las rutas privadas anidadas bajo este layout y una única envoltura de `RutaProtegida`.

```
<Route element={<RutaProtegida><MainLayout /></RutaProtegida>}>
  <Route path="/" element={<Dashboard />} />
  <Route path="/clientes" element={<ListaClientes />} />
  <Route path="/clientes/:id" element={<DetalleCliente />} />
</Route>
<Route path="/login" element={<Login />} />
<Route path="*" element={<ErrorPage />} />
```

✓ **Beneficio:** Desacopla la vista de Login y 404 del menú administrativo; elimina repetición de código protector en cada ruta.

2. Configuración de Path Aliases (`@/\*`) en Vite

Resuelve: HLG-15

Descripción: Añadir alias de resolución absoluta en `vite.config.js` vinculando el prefijo `@` a la carpeta `src/`.

```
export default defineConfig({
  plugins: [react()],
  resolve: { alias: { '@': path.resolve(__dirname, './src') } }
})
```

✓ **Beneficio:** Elimina importaciones frágiles y facilita el refactorizado de componentes en carpetas anidadas.

3. Rediseño y Jerarquía del Componente 404 (`ErrorPage`)

Resuelve: HLG-14

Descripción: Sustituir el encabezado plano por una tarjeta centrada con código de estado HTTP 404, descripción amigable del incidente y botón de retorno rápido al Dashboard principal.

✓ **Beneficio:** Impide que el usuario quede atrapado en una pantalla huérfana y mejora la coherencia visual del sistema.

B. Gestión de Estado y Hooks

4. Blindaje del Almacenamiento Persistente en Contexto

Resuelve: HLG-05

Descripción: Proteger la inicialización perezosa de `useState` dentro de `AutorizacionesContext.jsx` mediante `try...catch`.

```
const [admin, setAdmin] = useState(() => {
  try {
    const guardado = localStorage.getItem('admin');
    return guardado ? JSON.parse(guardado) : null;
  } catch (err) {
    localStorage.removeItem('admin');
    return null;
  }
});
```

✓ **Beneficio:** Previene caídas totales de la aplicación por datos corruptos o valores manipulados en el navegador.

5. Unificación del Rol en la Fuente Única de Verdad (Contexto)

Resuelve: HLG-02

Descripción: Suprimir la clave redundante `role` en `localStorage` desde `Login.jsx`. En `DetalleCliente.jsx`, consumir la propiedad de autorización directamente desde `useAutorizaciones()`: `admin?.sector === "Gerencia"`.

✓ **Beneficio:** Erradica la desincronización de roles y asegura que al cerrar sesión el acceso a funciones restringidas quede inmediatamente revocado.

6. Sincronización Inmediata entre Alta y Grilla de Clientes

Resuelve: HLG-06

Descripción: Parametrizar `FormCliente` para recibir la prop `onClienteCreado(nuevoCliente)`. Al completar exitosamente la petición POST, el formulario invoca este callback y `ListaClientes` anexa el registro al inicio del estado `clientes`.

✓ **Beneficio:** Proporciona reactividad instantánea en la interfaz sin requerir recargas de página.

7. Validación de Contexto en Hook Personalizado (`useAutorizaciones`)

Resuelve: Robustez de Hooks

Descripción: Incorporar una cláusula de guardia en `useAutorizaciones.js` que arroje un error explícito si el hook se invoca fuera del árbol provisto por `AutorizacionesProvider`.

✓ **Beneficio:** Facilita la depuración en tiempo de desarrollo ante errores de posicionamiento de componentes.

C. Servicios, Integración HTTP y Seguridad

8. Centralización Completa del Consumo de API con Axios

Resuelve: HLG-03

Descripción: Crear una instancia base de Axios (`api.js`) con `timeout` y `baseUrl`. Migrar todos los llamados dispersos en `ListaClientes` y `DetalleCliente` a funciones del servicio `clientesService` (`getClientes`, `getClienteById`, `crearCliente`, `eliminarCliente`).

✓ **Beneficio:** Elimina URLs repetidas en duro y desacopla la capa de presentación de la infraestructura de red.

9. Cancelación de Peticiones Asíncronas con `AbortController`

Resuelve: HLG-07

Descripción: Conectar la señal de aborto nativa (`signal`) en las llamadas de carga dentro de `useEffect` y ejecutar `controller.abort()` en la función de limpieza al desmontar.

✓ **Beneficio:** Previene fugas de memoria y actualizaciones de estado sobre componentes desmontados durante navegación rápida.

10. Ocultamiento de Credenciales y Simulación Segura de Autenticación

Resuelve: HLG-01

Descripción: Remover las credenciales en texto plano del bundle cliente. En `DetalleCliente.jsx`, suprimir la impresión visible de contraseñas o enmascararlas como asteriscos fijos.

✓ **Beneficio:** Cumplimiento de directrices elementales de ciberseguridad y privacidad de datos de usuarios.

D. Calidad de Código, Estilos y Experiencia de Usuario

11. Corrección Exhaustiva de Errores de ESLint y Fast Refresh

Resuelve: HLG-04

Descripción: Purgar variables huérfanas en `App.jsx`, `routes.jsx` y `DetalleCliente.jsx`. Ajustar la exportación en `AutorizacionesContext.jsx` para que únicamente exporte componentes React en el archivo del Provider.

✓ **Beneficio:** Pipeline de integración continua (CI) verde y soporte de recarga rápida en caliente (Vite HMR) sin reseteo de estado.

12. Aislamiento de Estilos mediante CSS Modules (`\*.module.css`)

Resuelve: HLG-09

Descripción: Transformar las hojas de estilo de login, lista de clientes y formulario a módulos CSS locales. Reemplazar selectores generales de etiqueta (`form`, `table`, `thead`) por nombres de clase con prefijo de ámbito.

✓ **Beneficio:** Erradica la contaminación de reglas globales y la necesidad de recurrir a `!important`.

13. Modal de Confirmación para Acciones Destructivas

Resuelve: HLG-12

Descripción: Integrar un componente `Modal` de React-Bootstrap en `DetalleCliente.jsx` que solicite confirmación expresa antes de ejecutar la eliminación física del cliente.

✓ **Beneficio:** Evita la pérdida accidental de registros por pulsaciones involuntarias.

14. Búsqueda y Filtrado Tolerante a Datos Nulos (Safe Navigation)

Resuelve: HLG-08

Descripción: Blindar la función de filtrado en `ListaClientes.jsx` utilizando encadenamiento opcional y fallback a cadenas vacías: `(cliente?.name?.lastname ?? '').toLowerCase().includes(busqueda)`.

✓ **Beneficio:** Garantiza que la interfaz no colapse si un registro proveniente de la API omite campos esperados.

15. Dinamización Real de Indicadores del Dashboard

Resuelve: HLG-11

Descripción: Vincular las tarjetas métricas del Dashboard a la consulta de clientes del servicio para computar el conteo real (`clientes.length`), y suprimir el bloque de login redundante.

✓ **Beneficio:** Transforma el panel de control estático en una herramienta analítica real y coherente.